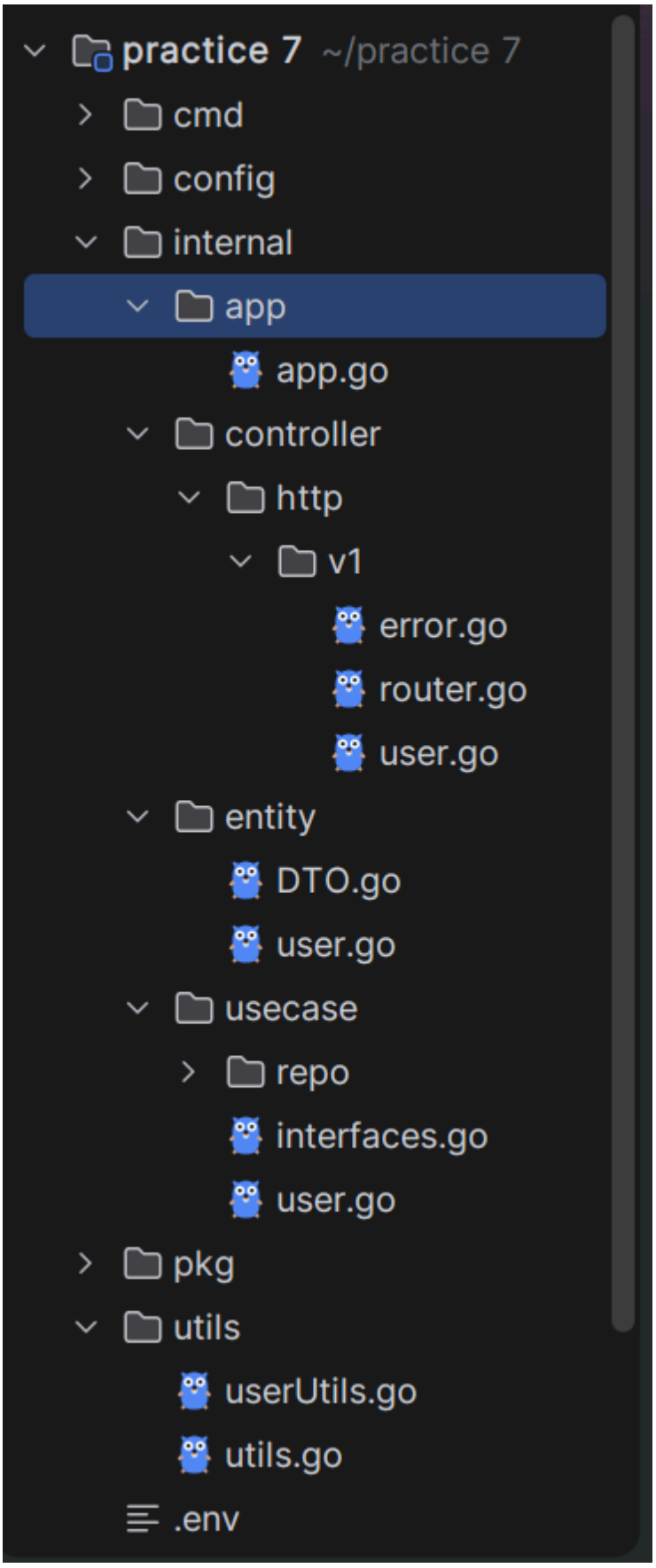
Before start, make sure you've read the following articles and sources:

- `x/crypto/bcrypt`: official documentation
- `golang-jwt`: official documentation
- Authentication and Authorization with Go Language with JWT.

> Below is my implementation of authentication and authorization logic. It may differ from the one I've provided above.

Structure of my project:



Step 1: Define User struct:

`internal/entity/user.go`

```
package entity

import (
```

```go
    "github.com/google/uuid"
)

type User struct {
    ID         uuid.UUID
`json:"ID";type:uuid;default:uuid_generate_v4()"`
    Username   string
    Email      string
    Password   string
    Role       string    // user,admin, etc.
    Verified   bool
}
```

> **Note: This is just my example. You can have your own User struct.**

Step 2: Expose an endpoint to register/create user:

`internal/controller/http/v1/user.go`:

```go
package v1

import (
    "net/http"
    "practice-7/internal/entity"
    "practice-7/internal/usecase"
    "practice-7/pkg/logger"
    "practice-7/utils"

    "github.com/gin-gonic/gin"
)

type userRoutes struct {
    t usecase.UserInterface
    l logger.Interface
}

func newUserRoutes(handler *gin.RouterGroup, t usecase.UserInterface, l
logger.Interface) {
    r := &userRoutes{t, l}

    h := handler.Group("/users")
    {
        h.POST("/", r.RegisterUser)
    }
}

...

func (r *userRoutes) RegisterUser(c *gin.Context) {
    var createUserDTO entity.CreateUserDTO
    if err := c.ShouldBindJSON(&createUserDTO); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
```

```go
    }

    hashedPassword, err := utils.HashPassword(createUserDTO.Password)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Error
hashing password"})
        return
    }

    user := entity.User{
        Username: createUserDTO.Username,
        Email:    createUserDTO.Email,
        Password: hashedPassword,
        Role:     "user",
    }

    createdUser, sessionID, err := r.t.RegisterUser(&user)
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})
        return
    }

    c.JSON(http.StatusCreated, gin.H{
        "message":    "User registered successfully. Please check your
email for verification code.",
        "session_id": sessionID,
        "user":       createdUser,
    })
}
```

`internal/entity/DTO.go`

```go
type CreateUserDTO struct {
    Username string `json:"username" binding:"required"`
    Email    string `json:"email" binding:"required,email"`
    Password string `json:"password" binding:"required,min=6"`
    Role     string `json:"role"` // Optional: "user" (default) or
"admin"
}
```

Step 2.1: Create utils folder in the root of your project:

Define HashPassword func:

`utils/utils.go`

```go
package utils

import (
    "net/http"
    "os"
    "strconv"
    "strings"
```

```go
	"time"

	"github.com/gin-gonic/gin"
	"github.com/golang-jwt/jwt/v5"
	"github.com/google/uuid"
	"golang.org/x/crypto/bcrypt"
)

var jwtSecret = []byte(os.Getenv("JWT_SECRET"))

func HashPassword(password string) (string, error) {
	bytes, err := bcrypt.GenerateFromPassword([]byte(password),
bcrypt.DefaultCost)
	return string(bytes), err
}
```

Step 2.2: Define interfaces:

`internal/usecase/interfaces.go`:

```go
package usecase

import (
	"practice-7/internal/entity"

	"github.com/google/uuid"
)

type (
	UserInterface interface {
		RegisterUser(user *entity.User) (*entity.User, string, error)
	}
)
```

Step 2.3: Implement UserInterface for `UserUseCase`

`internal/usecase/user.go`

```go
package usecase

import (
	"fmt"
	"practice-7/internal/entity"
	"practice-7/internal/usecase/repo"
	"practice-7/utils"

	"github.com/google/uuid"
)

type UserUseCase struct {
	repo *repo.UserRepo
}
```

```go
// NewTourismUseCase -.
func NewUserUseCase(r *repo.UserRepo) *UserUseCase {
	return &UserUseCase{
		repo: r,
	}
}

func (u *UserUseCase) RegisterUser(user *entity.User) (*entity.User,
string, error) {
	user, err := u.repo.RegisterUser(user)
	if err != nil {
		return nil, "", fmt.Errorf("register user: %w", err)
	}

	sessionID := uuid.New().String()
	return user, sessionID, nil
}
```

Step 2.4:

`internal/usecase/repo/user_repo.go`

```go
package repo

import (
	"fmt"
	"log"
	"practice-7/internal/entity"
	"practice-7/pkg/postgres"

	"github.com/google/uuid"
)

type UserRepo struct {
	PG *postgres.Postgres
}
func NewUserRepo(pg *postgres.Postgres) *UserRepo {
	return &UserRepo{pg}
}

func (u *UserRepo) RegisterUser(user *entity.User) (*entity.User, error) {
	err := u.PG.Conn.Create(user).Error
	if err != nil {
		return nil, err
	}
	return user, nil
}
```

`pkg/postgres`:

```go
type Postgres struct {
	Conn *gorm.DB
}
```

Step 3: Start your server

Make the http request for created endpoint:

POST http://localhost:8090/v1/users/

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
    "username": "Bakytzhan",
    "email": "bakytzhan@mail.com",
    "password": "12345678"
}
```

Body Cookies Headers (3) Test Results 201 Created

JSON Preview Visualize


```
{
    "message": "User registered successfully. Please check your email for verification code.",
    "session_id": "e0a8146c-fb2e-43d2-b59a-2a9dc42e8f23",
    "user": {
        "CreatedAt": "2026-04-01T15:08:32.773477939+05:00",
        "UpdatedAt": "2026-04-01T15:08:32.773477939+05:00",
        "DeletedAt": null,
        "ID": "97606770-6945-422e-855f-2e8bf8e72486",
        "Username": "Bakytzhan",
        "Email": "bakytzhan@mail.com",
        "Password": "$2a$10$3L2EyE3U5UPZ98.mXcj3DeiB3De4NxB/LzV2ryiZOoccm.iDn59ba",
        "Role": "user",
        "Verified": false
    }
}
```


> Nice work!

Pay attention that we're returning password in hashed format, so that even after someone gets the sensitive data, they won't be able to use or read it.

WHERE ORDER BY

	id	username	email	password	created_
1	97606770-6945-422e-855f-2e8bf8…	Bakytzhan	bakytzhan@mail.com	$2a$10$3L2EyE3U5UPZ98.mXcj3Dei…	2026-04-01

postgres@localhost 1 of 10
- postgres 2 of 4
  - auth_service
    - tables 1
      - users
        - columns 9
        - keys 3
        - indexes 4
  - public

> And user record created in the database!

Step 4: Login flow

We've implemented the registration flow. Now, let's make it possible for users to log-in to our system.

Add new endpoint to `internal/controller/http/v1/user.go`

```
func (r *userRoutes) LoginUser(c *gin.Context) {
	var input entity.LoginUserDTO

	if err := c.ShouldBindJSON(&input); err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
		return
	}

	token, err := r.t.LoginUser(&input)
	if err != nil {
		c.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})
	}

	c.JSON(http.StatusOK, gin.H{"token": token})
}
```

`internal/entity/DTO.go`

```
type CreateUserDTO struct {...}

type LoginUserDTO struct { // <- Add this struct
	Username string `json:"username" binding:"required"`
	Password string `json:"password" binding:"required"`
}
```

> ### Should be looking something like that:

```
...
func newUserRoutes(handler *gin.RouterGroup, t usecase.UserInterface, l
logger.Interface) {
	r := &userRoutes{t, l}

	h := handler.Group("/users")
	{
		h.POST("/", r.RegisterUser)
		h.POST("/login", r.LoginUser)
	}
}

func (r *userRoutes) RegisterUser(c *gin.Context) {...}
```

```go
func (r *userRoutes) LoginUser(c *gin.Context) {...}
```

Step 4.1:

Add new interface func

`internal/usecase/interfaces.go`

```go
type (
    UserInterface interface {
        LoginUser(user *entity.LoginUserDTO) (string, error) // <- here
it is!
        RegisterUser(user *entity.User) (*entity.User, string, error)
    }
)
```

`internal/usecase/user.go`

```go
func NewUserUseCase(r *repo.UserRepo) *UserUseCase {...}

func (u *UserUseCase) LoginUser(user *entity.LoginUserDTO) (string, error)
{
    userFromRepo, err := u.repo.LoginUser(user)
    if err != nil {
        return "", fmt.Errorf("User From Repo: %w", err)
    }
    if !utils.CheckPassword(userFromRepo.Password, user.Password) {
        return "", fmt.Errorf("Check Password: %w", err)
    }
    token, err := utils.GenerateJWT(userFromRepo.ID, userFromRepo.Role)
    if err != nil {
        return "", fmt.Errorf("Generate JWT: %w", err)
    }
    return token, nil
}

func (u *UserUseCase) RegisterUser(user *entity.User) (*entity.User,
string, error) {...}
...
```

> **Here, I am retrieving user objects from database by `username`. After that, I am hashing the password from input to compare it with the hashed one from database by calling `utils.CheckPassword(...)` function**

> **If passwords are same, we can now generate `JWT` and return it as response.**

Step 4.2: Add func at repo layer:

`internal/usecase/repo/user_repo.go`

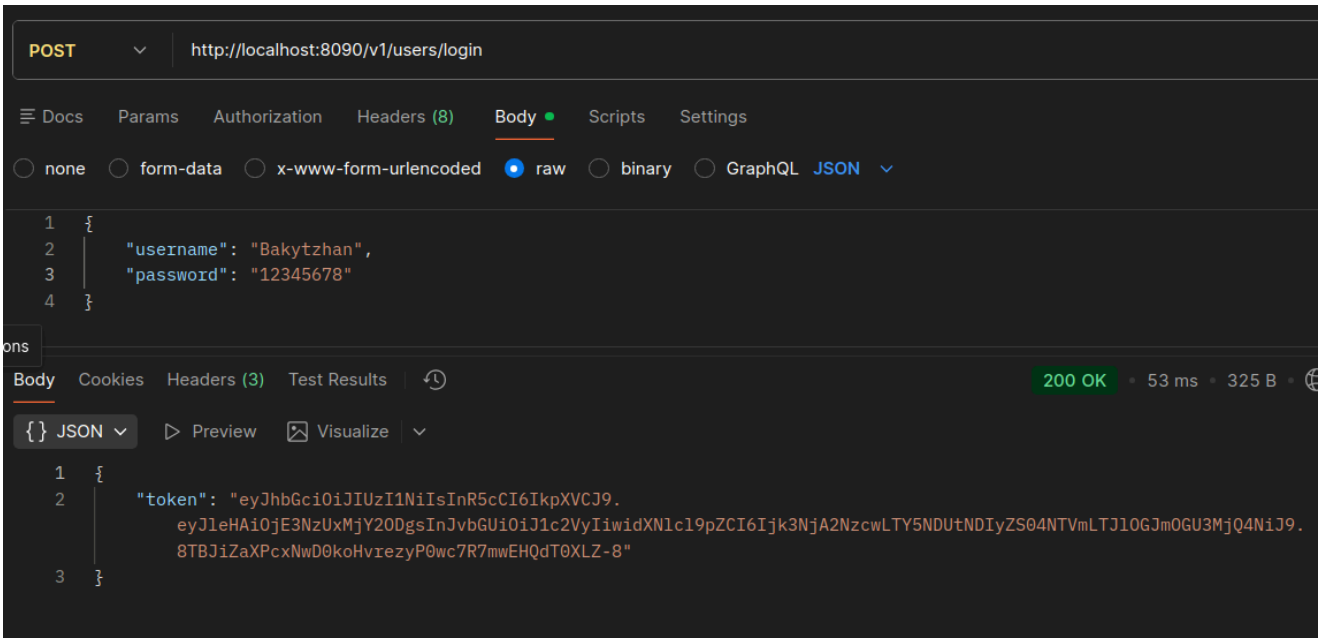
```
...

func NewUserRepo(pg *postgres.Postgres) *UserRepo {...}


func (u *UserRepo) LoginUser(user *entity.LoginUserDTO) (*entity.User,
error) {

     var userFromDB entity.User

     if err := u.PG.Conn.Where("username = ?",
user.Username).First(&userFromDB).Error; err != nil {
          return nil, fmt.Errorf("Username Not Found: %v", err)
     }
     return &userFromDB, nil
}

func (u *UserRepo) RegisterUser(user *entity.User) (*entity.User, error)
{...}
```

`utils/utils.go`

```
package utils

import (
     "net/http"
     "os"
     "strconv"
     "strings"
     "time"

     "github.com/gin-gonic/gin"
     "github.com/golang-jwt/jwt/v5"
     "github.com/google/uuid"
     "golang.org/x/crypto/bcrypt"
)

var jwtSecret = []byte(os.Getenv("JWT_SECRET"))

func HashPassword(password string) (string, error) {
     bytes, err := bcrypt.GenerateFromPassword([]byte(password),
bcrypt.DefaultCost)
```

```go
    return string(bytes), err
}

func CheckPassword(hashedPassword, password string) bool {
    return bcrypt.CompareHashAndPassword([]byte(hashedPassword),
[]byte(password)) == nil
}

func GenerateJWT(userID uuid.UUID, role string) (string, error) {

    claims := jwt.MapClaims{
        "user_id": userID,
        "role":    role,
        "exp":     time.Now().Add(time.Hour * 24).Unix(),
    }
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString(jwtSecret)
}
```

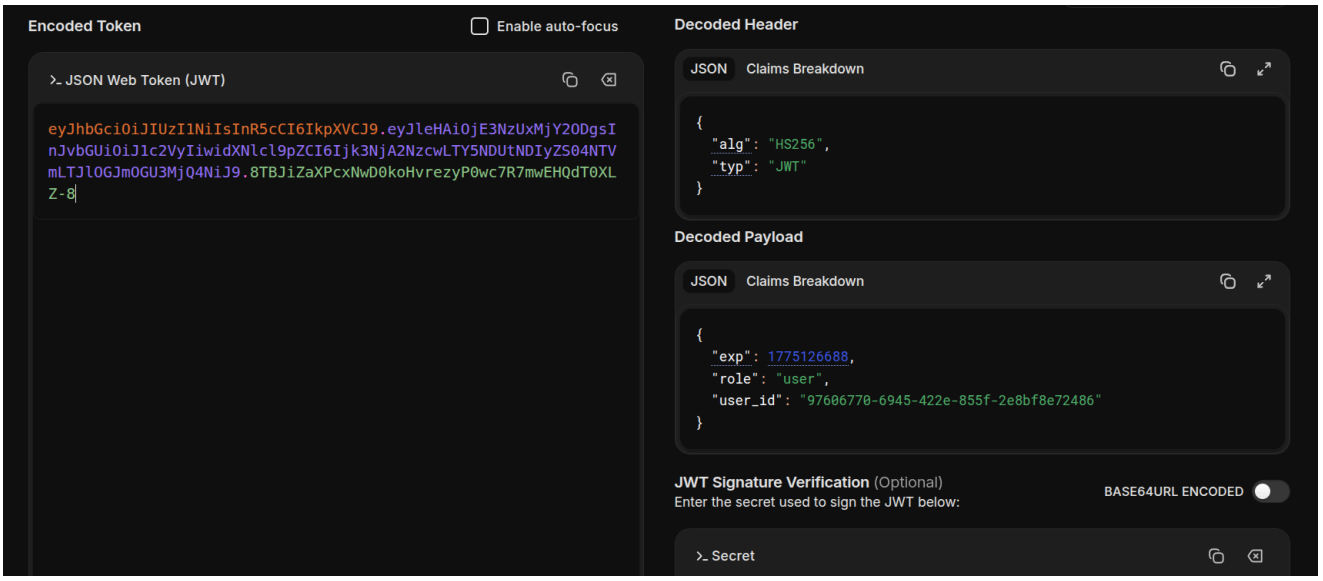
> **Well, let't test the logic:**



> **Great job!**

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3NzUxMjY2ODgsInJvbGUiOiJ1c
2VyIiwidXNlcl9pZCI6Ijk3NjA2NzcwLTY5NDUtNDIyZS04NTVmLTJlOGJmOGU3MjQ4NiJ9.8T
BJiZaXPcxNwD0koHvrezyP0wc7R7mwEHQdT0XLZ-8
```

If we paste this token to `jwt.io` we can get the contents of the token:



> **Try it yourself!**
>
> **Here, we can see the fields we've written to token before creating it: `exp`, `role`, and `user_id`**

Our system can register users and generate tokens based on credentials provided.

Now, let's use the token itself.

Step 5: implement JWTAuthMiddleware() func

`utils/utils.go`

```
func JWTAuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        tokenStr := c.GetHeader("Authorization")
        if tokenStr == "" {
            c.AbortWithStatusJSON(http.StatusUnauthorized,
gin.H{"error": "Token required"})
            return
        }

        tokenStr = strings.TrimPrefix(tokenStr, "Bearer ")

        token, err := jwt.Parse(tokenStr, func(token *jwt.Token)
(interface{}, error) {
            return jwtSecret, nil
        })

        if err != nil || !token.Valid {
            c.AbortWithStatusJSON(http.StatusUnauthorized,
gin.H{"error": "Invalid token"})
            return
```

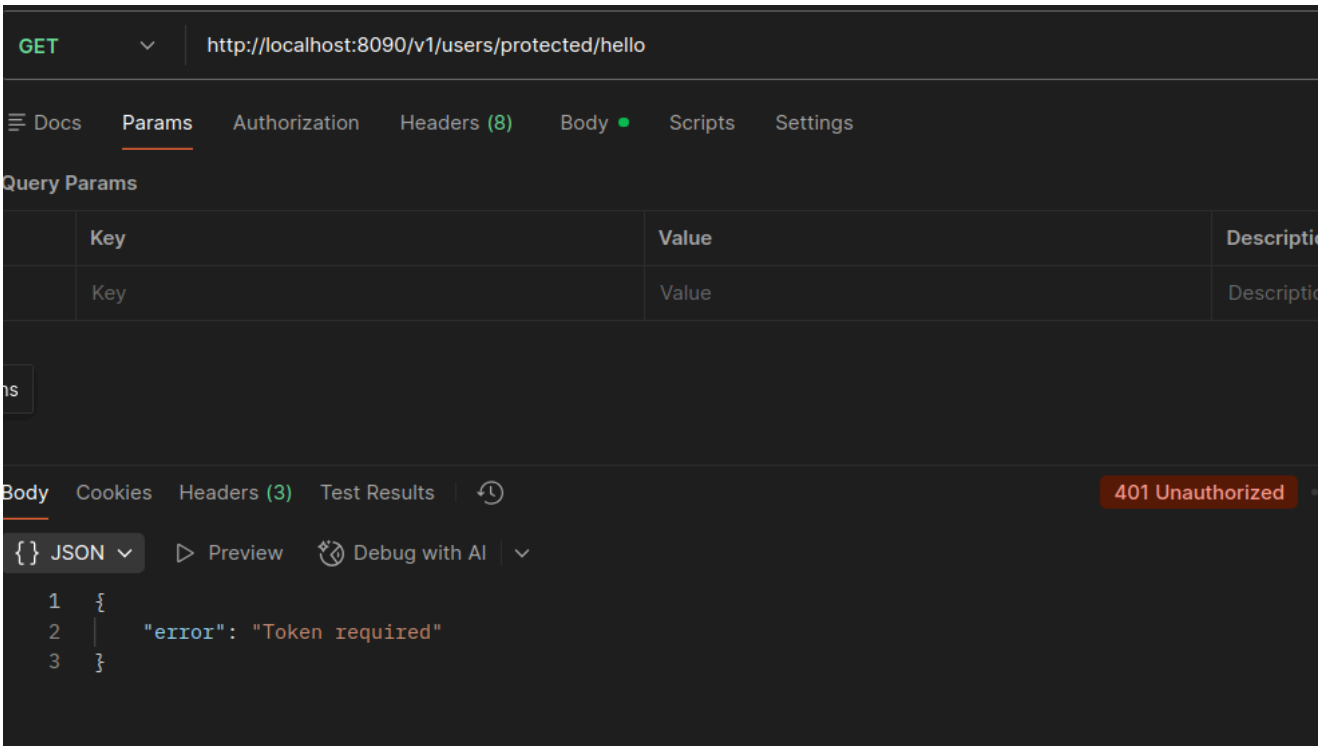
```
        }

        claims, _ := token.Claims.(jwt.MapClaims)

        c.Set("userID", claims["user_id"].(string))
        c.Set("role", claims["role"].(string))
        c.Next()
    }
}
```

Step 5.1: Define a function that will work through middleware:

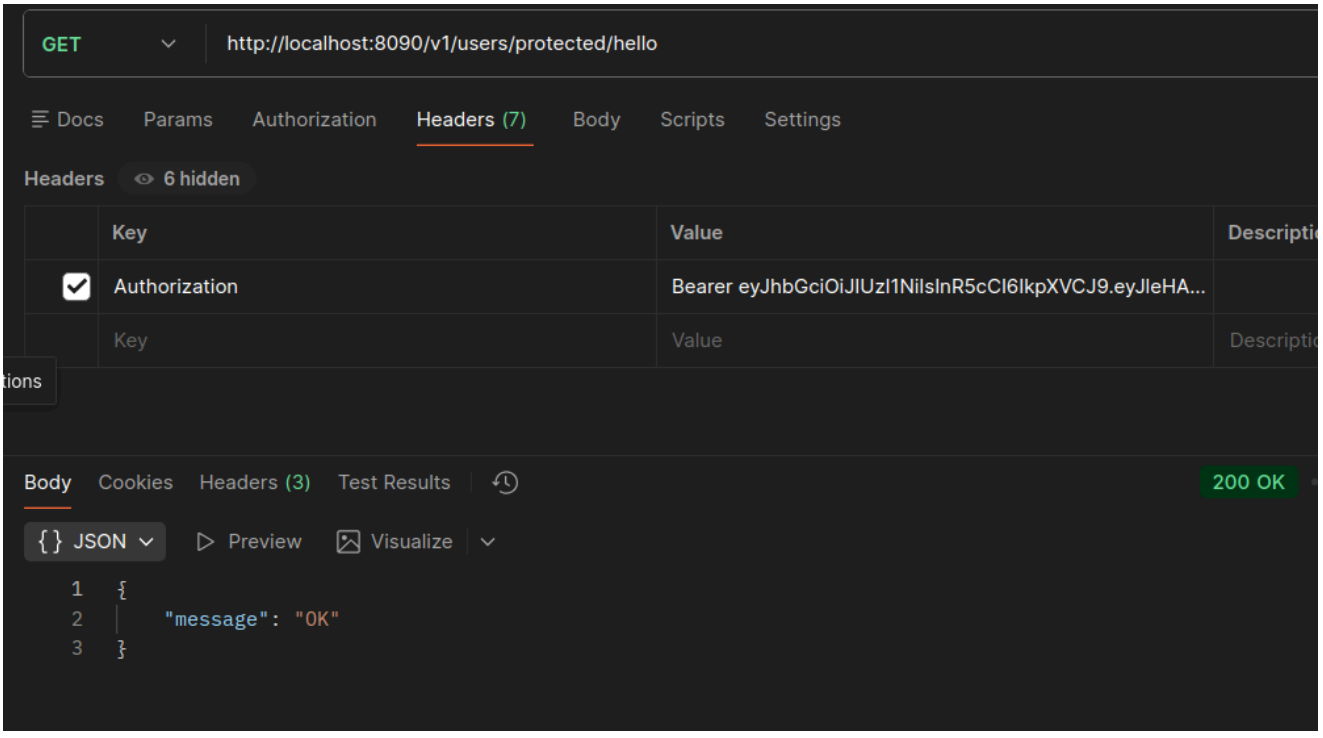
`internal/controller/http/v1/user.go`

```
...
func newUserRoutes(handler *gin.RouterGroup, t usecase.UserInterface, l
logger.Interface) {
    r := &userRoutes{t, l}

    h := handler.Group("/users")
    {
        h.POST("/", r.RegisterUser)
        h.POST("/login", r.LoginUser)

        protected := h.Group("/")
        protected.Use(utils.JWTAuthMiddleware())
        {
            protected.GET("/protected/hello", r.ProtectedFunc)
        }
    }
}

func (r *userRoutes) ProtectedFunc(c *gin.Context) {
    c.JSON(200, gin.H{"message": "OK"})
}
...
```

Let's try to make request to this endpoint:



Now, with token:



> ## Much better!

Individual Assignment (3 points):

1. Define `GetMe(c *gin.Context)` function that returns the information about the user:

Requirements:

- Function should work through `JWTAuthMiddleware()`

- No data should be provided except the JWT token in http request to this function.

- Register User A and User B. Show that calling `/users/me` with User A's token returns User A's email, and User B's token returns User B's email.

2. Authorization and Role-Based Access

- Create a "Promote User" endpoint that changes the role of a specific user from `user` to `admin`.

Requrements:

- Create a new Middleware: `RoleMiddleware(requiredRole string)`.

- This middleware should check the `role` claim stored in the JWT.

- Only users with the role `admin` should be able to access the endpoint `PATCH /users/promote/:id`

3. Implement `Rate Limiter` functionality

Implement a middleware that limits the number of requests a user (or IP address) can make to your API within a specific timeframe. If the limit is exceeded, the server should return a 429 Too Many Requests error.

Requirements:

Your Rate-Limiter middleware must support two models of identification:

- Authenticated: Use the `UserID` from JWT

- Anonymous: Use the `ClientIP` from the Gin context if no JWT is provided

Use `sync.Mutex` to prevent race conditions.

Demo video scenario:

- Student runs golang application
- Student demonstrates:
  - IN **POSTMAN**:
    - `GetMe()` function with 2 users provided according to the requirements
    - Role-Based Access for the task №2
    - Rate-Limiter Functionality

Definition of Done:

- The backend service is running successfully
- All 3 tasks are accomplished according to the requirements
- `Demo Video` is recorded according to the `Demo Video Scenario`

Deliverables:

- Push all the changes to your GitHub repository
- Submit the link to this repository(Please, submit the link as a LINK ATTACHMENT, not a file)
- Submit [1-2]-minute demo video of your project as a VIDEO-FILE ATTACHMENT. You can speed it up if it is too long.

> ## Below are marking criteria:

Criteria	Weight
Problem 1	0.75 points
Problem 2	0.75 points

Criteria	Weight
Problem 3	1.5 points
OVERALL	3 points

Below are penalties criteria:

Adherence to all the requirements above is mandatory. Failure to implement any of the specified points (e.g., missing `sync.WaitGroup` or neglecting `context` support) will result in a significant point deduction.

Category	Criteria	Penalty
Video Demo	Video Demo is does not Present or Unable to Open/View or Contradicts ethical standards	3 points
Video Demo Link	The video demo is specified in any way other than the `video-file` attachment	0.5 points
Git Repo Link	The link to the git repository is specified in any way other than the `link-attachment`	0.5 points
Git Repo Content	The content of git repository for this practice is corrupted or unable to Open/Read	3 points
Your Task part	At least one requirement (e.g. Demonstrate the execution of the code using -race flag.) from `Individual Assignment` part is failed to meet	100% of a problem's solution mark
Cheating/ Malpractice	Students practicing cheating, duplicating the contents of other students solutions	100% of the `Overall` mark for each of the students

---

DEADLINE: 12.04.2026, SUNDAY 23:59

OPTIONAL FEATURES (Choose Freely)

> **Students may implement any number of the following to increase difficulty and engagement. Your optional features will be considered only if you demonstrated them in your demo video and explained it in details.**

🐣 EASY: "The Baby Gopher"

- Implement email verification flow by sending 4-digit code to the actual email address provided by user.
- User then sends your code and his account becomes `verified`
- Limit access to `unverified` users

👨‍💻 MEDIUM: "The Gopher-at-Work"

- Refactor the login flow to return two tokens: an Access Token (expires in 15 minutes) and a Refresh Token (expires in 7 days).
  Requirements:
  - Modify `GenerateJWT` to produce both.
  - Create a new endpoint `POST /users/refresh`.
  - The Logic: This endpoint should accept the Refresh Token, verify it, and issue a *new* Access Token.

🧙‍♂️ ADVANCED: "The Gopher Wizard"

- Shift your authentication from purely stateless to a hybrid approach. When a user logs in, you will store their active token in Redis. To log out, you simply delete that record. Your middleware will now check Redis to see if the token is still "active."
  Requirements:
  - Setup and use Redis
  - After calling `utils.GenerateJWT`, you must save this token to Redis.
  - Implement a new endpoint: `POST /users/logout`.
    - Delete the key `auth:user_uuid` from Redis.
    - Do not accept the active token anymore even if the JWT is technically still "valid" by its own timer, the middleware rejects it because it's missing from Redis.